

MOVIE RECOMMENDATION SYSTEM

Internship Project Report

A Content-Based Filtering Approach Using TF-IDF and Cosine Similarity

Technology: Python | Pandas | scikit-learn | Joblib

Prepared for Internship Submission

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Technologies Used
6. System Design and Methodology
7. Implementation
8. Results and Discussion
9. Advantages and Limitations
10. Future Scope
11. Conclusion
12. Viva Questions and Answers
13. Project Structure
14. How to Run

1. Abstract

A movie recommendation system helps users discover films that match their interests. This project implements a simple content-based recommendation system in Python. Each movie is represented by its genre, keywords and description. Text is converted into numerical vectors with TF-IDF, and cosine similarity is used to compare movies. For a selected movie, the system returns the five most similar movies along with genre, rating and similarity score. The project is intentionally small and transparent so that the complete workflow can be understood and demonstrated easily.

2. Introduction

Recommendation systems are widely used in digital platforms to help users find relevant content. A content-based system recommends items that are similar to an item the user already likes. Unlike collaborative filtering, it does not require a large history of users and ratings. This makes content-based filtering suitable for a small academic demonstration.

3. Problem Statement

A user may know the name of one movie they enjoy but may not know what to watch next. The goal of this project is to build a program that takes a movie title as input and recommends similar movies automatically.

4. Objectives

- Create a small structured movie dataset for demonstration.
- Represent movie information as text features.
- Convert text into TF-IDF vectors.
- Measure similarity between movies using cosine similarity.
- Display the top five recommendations interactively.
- Provide a simple runnable project suitable for internship demonstration.

5. Technologies Used

Technology	Purpose	Why Used
Python	Programming language	Simple and widely used for data science
Pandas	Dataset handling	Reads and processes CSV data
scikit-learn	Machine learning	Provides TF-IDF vectorizer and similarity tools
Joblib	Model persistence	Saves the processed vectorizer and matrix
CSV	Data storage	Easy to inspect and edit for an academic project

6. System Design and Methodology

1. Step 1 – Data collection: The bundled dataset contains 40 movies with title, genre, keywords, description and rating.
2. Step 2 – Feature preparation: Genre and keywords are converted to text and combined with the description.
3. Step 3 – TF-IDF: Each movie is converted into a weighted vector based on words and two-word phrases.
4. Step 4 – Cosine similarity: Similarity is computed between the selected movie vector and all other movie vectors.
5. Step 5 – Ranking: Movies are sorted by similarity score and the top five are displayed.
6. Step 6 – User interaction: The terminal accepts a movie title, prints recommendations, supports a title list, and exits on quit.

6.1 TF-IDF

TF-IDF stands for Term Frequency–Inverse Document Frequency. It gives greater weight to words that are useful for distinguishing one movie from another, while reducing the influence of words that occur broadly across many descriptions. In this project, TF-IDF turns movie text into numerical feature vectors.

6.2 Cosine Similarity

Cosine similarity compares the angle between two vectors. A value closer to 1 indicates that the feature vectors point in a more similar direction. The system uses these scores to rank candidate movies.

7. Implementation

Core Python logic:

```
vectorizer = TfidfVectorizer(stop_words="english", ngram_range=(1, 2))
matrix = vectorizer.fit_transform(df["features"])
similarities = cosine_similarity(matrix[idx], matrix).flatten()
ranked = similarities.argsort()[::-1]
```

The project separates data, source code, saved artifacts and documentation. The main executable file is `code/movie\_recommender.py` and the dataset is `data/movies.csv` in the project folder.

8. Results and Discussion

A sample run using the bundled dataset produced the following recommendations:

Input Movie	Rank	Recommendation	Similarity
Inception	1	Interstellar	7.67%
Inception	2	Avengers: Endgame	7.21%
Inception	3	Guardians of the Galaxy	4.65%
The Dark Knight	1	John Wick	7.96%
The Dark Knight	2	Iron Man	5.66%

Toy Story	1	The Intouchables	10.71%
-----------	---	------------------	--------

The results demonstrate the intended behavior: movies sharing important terms, genres or themes with the selected title tend to receive higher similarity scores. The score is a text-feature similarity rather than a prediction of user satisfaction.

9. Advantages and Limitations

Advantages

- Simple to understand and demonstrate.
- No large user database is required.
- Recommendations are explainable through text similarity.
- Works offline with the bundled dataset.

Limitations

- The dataset is small and created for academic demonstration.
- It does not learn from real user behavior or ratings.
- Recommendations depend on the quality of the movie descriptions and keywords.
- It cannot capture a user's personal preferences beyond the selected movie.

10. Future Scope

- Use a larger real-world movie dataset.
- Add user ratings and build a hybrid recommender.
- Add collaborative filtering.
- Develop a web interface using Flask or Streamlit.
- Add filters for genre, language, year and rating.
- Evaluate recommendations with user-based metrics such as Precision@K and Recall@K.

11. Conclusion

The Movie Recommendation System demonstrates a complete introductory recommendation workflow. The project combines text preprocessing, TF-IDF feature extraction and cosine similarity to produce ranked movie recommendations. Because the implementation is small and transparent, it is suitable for learning, internship submission and a basic live demonstration.

12. Viva Questions and Answers

Q: What is a recommendation system?

A: A system that suggests relevant items to users based on item features, user behavior, or both.

Q: Which recommendation technique is used?

A: Content-based filtering.

Q: Why is TF-IDF used?

A: To convert text into numerical feature vectors while giving useful importance to words.

Q: What is cosine similarity?

A: A measure of similarity between vectors based on the cosine of the angle between them.

Q: Why not use collaborative filtering?

A: It would require user-item interaction data; this project intentionally uses only movie content.

Q: What is the input?

A: A movie title typed by the user.

Q: What is the output?

A: The five most similar movies with genre, rating and similarity score.

Q: Is the system production-ready?

A: No. It is an academic demo with a small bundled dataset.

13. Project Structure

```
Movie_Recommendation_System/  
|-- code/  
|   |-- movie_recommender.py  
|   `-- train_model.py  
|-- data/  
|   `-- movies.csv  
|-- artifacts/  
|-- docs/  
|-- slides/  
|-- requirements.txt  
|-- README.md  
`-- run_project.bat
```

14. How to Run

Open a terminal in the project folder and run:

```
pip install -r requirements.txt  
python code/train_model.py  
python code/movie_recommender.py
```

Then type a movie title such as `Inception`, `Toy Story`, or `The Dark Knight`. Type `list` to view all available titles and `quit` to exit.